

CSCI 412: Senior Seminar

Project Proposal

Brandon Terry

February 4, 2026

Project Title

Automated Network Traffic Analysis & Suricata IDS Evaluation Toolkit

Problem Description and Motivation

Modern technologies and networks are becoming more advanced and expansive, making the protection of data and resources even more crucial in today's times. Security analysts often need to move between various tools and analyze large data sources like packet captures, system logs, and IDS alerts. From my previous project experience, even in a lab setting, these workflows can become time-consuming as they require manual setup and work. As a result, the same analysis can take different amounts of time and even produce inconsistent outputs, which makes it harder to reproduce and explain.

The goal of this project is to make network traffic detection and analysis faster, more repeatable, and consistent through scripting and automation. We aim to achieve this by developing a small toolkit that will automate traffic capture, label attack windows, run IDS evaluations, and generate a structured summary of it all. By reducing the manual work required and enforcing more consistent procedures, the toolkit will be able to reduce the duration it takes to triage alerts while also helping users build intuition for recognizing attack patterns that produce valid evidence to be highlighted and documented.

Final Product Description

The final product will be a fully documented Network Traffic Analysis & IDS Evaluation Toolkit hosted on GitHub. The toolkit will provide an automated, reproducible method for generating baseline and malicious network traffic in a virtual lab environment. It will be able to capture that traffic into the PCAP dataset while running IDS detection with Suricata. All of it helps to extract useful traffic and alert features to produce an organized summary for comparison and evaluation.

The system will include scripts that complete common tasks, including starting and stopping captures, running attack simulations, labeling attack windows, processing packet captures and

logs, and generating reports. The GitHub will include setup instructions and reproducible run commands.

System Architecture

The project uses a virtual lab designed to support repeated traffic generation, capture, detection, and analysis.

- **Attack Machine Layer (Kali Linux VM):** Generate malicious traffic, i.e., SSH brute force attempts and network scanning.
- **Target Machine Layer(Ubuntu Linux VM):** Hosts network services and produces system logs that can support validation of attack activity.
- **Monitoring/Data Layer(Host Machine):** Captures network traffic using TShark/Wireshark, runs Suricata for IDS evaluation, and executes automation scripts for parsing and reporting.

The organized pipeline:

1. Kali VM (attacker) ---> Ubuntu VM (target)
2. Capture (TShark/Wireshark) + Suricata (IDS)
3. Python Analysis + Report Generation
4. Outputs: alerts, features, comparison summaries

Platform and Technologies

- **Platform:** Virtual lab environment and host-based automation toolkit
 - Host Platform: macOS
 - Virtual Platforms: VirtualBox and UTM
- **Operating Systems:**
 - Attacker VM: Kali Linux via VirtualBox
 - Target VM: Ubuntu Linux via UTM (SSH enabled)
 - Host OS: macOS
- **Programming Languages:**
 - Python (scripts and parsing)
 - Bash (scripts)
- **Tools and Frameworks**
 - Packet Capture: Wireshark TShark
 - Instruction Detection/Alerting: Suricata
 - Attack Simulation: Hydra & Nmap
 - Data Processing & Visualization: Python
 - Version Control & Documentation: Git and GitHub

Core Functionalities

The toolkit will implement the following core features:

- **Scenario Automation (Repeatable runs)**
 - Start capture, execute baseline or attack activity (SSH brute force and one additional if time permits), stop capture, save outputs.
- **Traffic Capture and Dataset Management**
 - Generate and store PCAP for each run and save run metadata
- **IDS Evaluation**
 - Run Suricata offline on collected PCAPs and produce EVE JSON alert logs
- **Feature Extraction and Summarization**
 - Parse PCAP/alerts to generate a structured summary comparing baseline vs attack
- **Alert Validation**
 - Record attack windows, compare alerts
- **Documentation and Reproducibility**
 - Provide clear instructions, consistent structure, and citations for resources.

Methodology

The project will be developed over time, starting with a basic automation pipeline, starting with the capture, then IDS evaluation and finally analytics output. Over time, it will be expanded in controlled steps. Initial development focus will be on stabilizing the VirtualBox/UTM lab setup, standardizing scenario scripts for both baseline and attack traffic simulation. Once the process is made sufficiently repeatable, Suricata will be integrated, followed by Python-based parsing. The final part will focus on tuning and creating custom rules, and refining documentation and deliverables. The progress will be tracked through GitHub commits, issues, and updates.

Testing and Validation

- **End-to-End Pipeline Testing:** Verify each scenario runs successfully from capture through summary output
- **Reproducibility Testing:** Re-run scenarios to confirm outputs are consistently generated, and results are comparable across runs
- **Ground Truth Validation:** Record known attack time windows and confirm Suricata alerts and indicators align
- **Baseline Noise/False Positive Check:** Verify detection rules produce minimal or no false positives on baseline traffic
- **Attack Detection Checks:** Confirm that attack scenarios reliably trigger intended alerts or indicators
- **Document Limitations:** Document testing outcomes, failures, and limitations

AI and External Resources

AI tools may be used to assist with debugging, scripting ideas, documentation drafting, and concept clarification during development. All AI-generated suggestions will first be reviewed and tested to ensure full understanding and the ability to explain all work I have completed. Any external resources used like open-source libraries, Suricata rule reference, or tutorials, will be properly cited in both the GitHub and the final written report, showing precisely what was used and how.

Timeline

- **Week 1-2:** Finalize scope, create GitHub, set up and verify stability of VirtualBox and UTM
- **Week 3-4:** Implement baseline structure automation and standardized PCAP and metadata outputs
- **Week 5-6:** Implement automated attack scenarios (SSH brute force)
- **Week 7-8:** Integrate Suricata PCAP evaluation and generate standardized alert output
- **Week 9-10:** Develop Python feature extraction and auto summary reporting
- **Week 11-12:** Implement and tune Suricata rules, document before/after results, and baseline noise checks
- **Week 13:** Full end-to-end testing and documentation
- **Week 14:** Final report and Presentation

Expected Deliverables

By the end of the semester, the following deliverables will be submitted:

- Automated Network Traffic Analysis & IDS Evaluation Toolkit
- Complete GitHub Repository with automation scripts, Suricata configuration, and citations
- Final Written Report
- Final Presentation and Demonstration

Expected Outcomes

The completed project will demonstrate the ability to design and implement a reproducible, automated security analysis pipeline. It will show practical skills in scripting, packet capture, and IDS evaluation. By the end of the semester, the toolkit and documentation will be portfolio-ready and suitable to present to employers as an example of workflow engineering and applied network security analysis.